

Econstellar: The Compute Engine

A Verifiable Econometrics Service: Architecture, Methods, and a GPU Upgrade

Avishek Bhandari

School of Humanities, Social Sciences and Management
Indian Institute of Technology Bhubaneswar

13 June 2026

Abstract

Empirical findings in financial econometrics are easy to publish and hard to re-run: the distance between a printed coefficient and a number a reader can reproduce on demand remains wide. We describe Econstellar’s compute engine, a small sandboxed service that narrows that distance by treating every published method as a re-runnable, self-checking object. A researcher selects a method and a data series — or asks a chatbot in plain language — and the engine executes the analysis inside an isolated sandbox, returns the result as structured data, and, for the methods that re-run a published study, reports how far the live number sits from the archived one. Twenty-six methods are in service, spanning unit-root and cointegration tests, volatility and long-memory estimators, transfer-entropy information flow, connectedness networks, contagion-channel attribution and a daily systemic-risk index; each ships with a pre-registered, *failable* evaluation, so a regression in any of them turns a public dashboard red rather than passing silently. We document the engine from first principles to its present form, including a recent episode in which the heaviest method saturated all twenty-two cores of its host, and the two-part remedy: a governed core budget, and a GPU port of the nearest-neighbour search that is bit-exact against the original estimator and runs the formerly host-hanging workload in thirty-six minutes on a single core. The contribution is less any individual number than the discipline around it — every figure in this manuscript resolves to a job identifier, a revision string, or a published artefact.

Keywords: reproducible econometrics; transfer entropy; systemic risk; sandboxed computation; GPU acceleration; financial networks.

JEL: C55, C58, C63, G01.

1 Introduction

There is a quiet asymmetry in applied econometrics. Producing a result is cheap; making that result re-runnable, by someone else, on demand, is expensive — and so it is mostly not done. A reader of a typical spillover or contagion paper is offered a table and an appendix, not a button. If a number is later questioned, the burden of reconstruction falls on whoever raises the question, and the original computation, with its package versions, its random seeds and its data vintage, is rarely recoverable in full.

Econstellar’s compute engine is our attempt to invert that asymmetry for a working research group. It is a small service, written in Node and R and built for the research platform of the School of Humanities, Social Sciences and Management at IIT Bhubaneswar, that does one thing carefully: it runs a fixed catalogue of econometric methods inside a sandbox and returns each result as structured data, with enough provenance attached that the number can be checked against an expectation and reproduced later. The design thesis is simple to state. *A method is not in the catalogue until it ships with a failable test of its own output.* The catalogue and that suite of tests are the deliverable; this manuscript is the manual.

We write for two readers at once. For the economist, the engine is a way to treat a published estimator as a live object — to ask “does this still hold” and get an answer banded against the paper’s own figure. For the software developer, it is a study in a particular kind of reliability: a service whose correctness is asserted by a pre-registered evaluation rather than asserted by its authors, and whose heaviest computation was recently moved from an unbounded CPU fork that hung its host onto a graphics processor, under a constraint that no published number was allowed to move by so much as the fifteenth decimal place. The sections that follow take the engine in order: what it is made of (§2), the methods it serves (§3), the verified record it has produced (§4), the GPU upgrade and the host-saturation episode that prompted it (§5), and the reproducibility discipline and nightly operation that hold the whole thing together (§6).

2 System architecture

The engine runs across three planes, each chosen for a different cost.

The kernel. The synchronous methods and the chatbot live in a container on Google Cloud Run (service `shssm-compute`, region `asia-south1`). Cloud Run scales to zero between requests, which suits a research service with bursty, low-volume traffic: most methods return in a few seconds, and the marginal cost of an idle engine is nothing. A request names a method and a parameter object; the kernel validates both against a fixed schema, runs the analysis, and returns one JSON document.

The tower. A handful of methods are too heavy for a request-scoped container — a transfer-entropy panel with surrogate resampling can run for half an hour. These route instead to an always-on job server on the group’s workstation (`server/job-server.mjs`, port 3030), which accepts a job, returns an identifier immediately, and runs the work asynchronously with a small fixed concurrency. The kernel and the dashboard poll for completion. Four methods currently take this path: the KSG transfer-entropy estimator and its robustness grid, the end-to-end NAMH pipeline, and the SOCH robustness badge.

The surfaces. The public read side is a set of static pages on GitHub Pages — a research dashboard, an evaluations page, a reproduction page, a claims ledger and a report gallery — together with one machine-written artefact, `evals.json`, which is the output of a single genuine run of the evaluation suite and the single source of truth the pages render. Nothing on the read side computes; it displays what the suite recorded.

Request lifecycle and the sandbox. Every analysis follows the same path: *method + params* → *schema validation* → *sandboxed Rscript* → *JSON on stdout*. The user never supplies code. They choose a method from a fixed registry and a set of schema-validated parameters, and the runner scripts are part of the repository, so there is no arbitrary-code-execution surface. Each analysis runs under `bubblewrap` (`bwrap`) with the network unshared, the filesystem mounted read-only, a fresh `tmpfs` scratch, and a wall-clock timeout; if `bwrap` is absent the runner degrades to the timeout alone. The data contract is deliberately narrow — one JSON object in, one JSON object out — which is what lets the same method run identically in the cloud container, on the workstation, and in the test harness.

The chatbot. A natural-language layer maps a question such as “*is China’s market stationary?*” through Gemini 2.5 Flash function calling onto the same validated registry the dashboard uses, runs the chosen method in the same sandbox, and returns the explained result. The chatbot adds no new computation; it is a front door onto the catalogue, not a parallel one.

3 The method library

The catalogue holds twenty-six methods. Six of them do something unusual for an analytics tool: rather than reimplement a published estimator, they call the *paper’s own package* and check the result. The NAMH methods run the `namh` R package [8]; the SOCH robustness badge runs the `sochcontagion` package’s own symmetry test; the contagion-channel attribution is anchored to the published decomposition (arXiv:2604.26546). This matters for trust: when the engine says a paper’s number reproduces, it is the paper’s code that produced the live number, not our paraphrase of it.

Table 1 groups the catalogue into six families. The estimators in the information-flow and connectedness families rest on a well-established literature, cited in the table; the stationarity, volatility and spectral families use standard implementations from the `urca`, `tseries` and `waveslim` packages and are named rather than re-derived. Transfer entropy deserves a word, because it is both the analytical core of the engine and the subject of §5. We estimate the information flow from one market to another [3] using the Kraskov–Stögbauer–Grassberger nearest-neighbour scheme [1] in the Frenzel–Pompe conditional-mutual-information form [2], on log-returns, which are $I(0)$ — never on price levels, which are $I(1)$. Significance is assessed against IAAFT surrogates [4], which preserve a series’ amplitude distribution and power spectrum while destroying its nonlinear structure, and the resulting network of directed links is thinned under Benjamini–Hochberg false-discovery control [5]. Connectedness is measured with the Diebold–Yilmaz forecast-error-variance decomposition [6], static and rolling.

Family	Methods	Published basis
Stationarity, cointegration & panels	<code>unit_root</code> , <code>live_unit_root</code> , <code>panel_unit_root</code> , <code>vecm</code> , <code>granger</code>	ADF/KPSS, Johansen, IPS; <code>urca</code> , <code>tseries</code>
Volatility, long memory & spectra	<code>garch</code> , <code>dfa_hurst</code> , <code>namh_hurst</code> , <code>wavelet</code> , <code>wavelet_coherence</code>	GARCH(1,1), DFA, MODWT variance/coherence; <code>waveslim</code>
Information flow (transfer entropy)	<code>ksg_te</code> , <code>ksg_robustness</code> , <code>namh_te</code> , <code>wqte</code> , <code>soch_profile</code>	KSG/Frenzel–Pompe [1, 2, 3]; IAAFT [4]; FDR [5]
Connectedness & networks	<code>connectedness</code> , <code>spillover_rolling</code> , <code>rolling_dcc</code> , <code>network</code> , <code>var_irf</code> , <code>quantile_var</code>	Diebold–Yilmaz [6]; DCC; VAR/IRF; quantile VAR; <code>igraph</code>
Contagion channels & systemic risk	<code>channel_attribution</code> , <code>sri_daily</code>	Channel attribution (arXiv:2604.26546); daily SRI
Reproduction & formal verification	<code>namh_reproduce</code> , <code>namh_pipeline</code> , <code>soch_robustness</code>	<code>namh</code> [8]; Δ -verifier; seeded badge [7]

Table 1: The twenty-six-method registry, grouped into six families. The synchronous kernel serves all but four methods; `ksg_te`, `ksg_robustness`, `namh_pipeline` and `soch_robustness` run as asynchronous tower jobs. The first three families are estimators; the last three include methods that re-run and check a published study rather than reimplement it.

4 The verified record

The engine’s claim is not that its numbers are impressive but that they are traceable. This section states the record of the most recent full verification run (2026-06-12), against which the present manuscript is anchored; every figure resolves to a job, a revision, or a published artefact.

The canonical run. The reference job for the NAMH pipeline is `job_20260612_0f2f0fd5`: a seeded end-to-end surrogate run (2,000-replication null, generator L’Ecuyer-CMRG, seed 42, 1,222 seconds of wall-clock). It returned a transfer-entropy window mean of -0.22322165 , inside its pre-registered band $[-0.2242, -0.2222]$. It also returned a network with zero edges surviving false-discovery control (0/552) — and we report that honestly, as a degenerate network, rather than relaxing the threshold to manufacture structure. An honest null is a result.

Substantive anchors. The run’s headline figures held without exception: the India ADF statistic of -49.18 ; GARCH(1,1) persistence $\alpha + \beta = 0.991$; a panel IPS statistic of -77.26 on the named market tuple {India, USA, UK, China, Japan}; the NAMH transfer-entropy window mean above; the KSG transfer entropy from the United States to Japan of 0.154234 ; the KSG robustness rank-stability Spearman of 0.7282 ; the global-financial-crisis trade-channel share of 27.9% (anchored to arXiv:2604.26546, read with that study’s own method-dependence disclosure); and a live systemic-risk index of 0.00849 . The panel statistic carries a cautionary lesson worth stating in a manual: -77.26 was unreproducible until the market set was *named*, since a different fifth market gives -80.16 . A documented figure that does not state its full parameterisation is not a reproducible claim.

Honest negatives. Two results are reported as not-significant and treated as immutable facts of the record. The SOCH-C significance test remains not significant at $p = 0.105$; the FDR-gated NAMH network is degenerate, as above. Neither is a blemish to be managed away. The reproduction layer underlines the point from the other direction: the Δ -verifier `namh_reproduce` re-runs the published estimators and reports a maximum absolute deviation from the archived output of 1.55×10^{-15} — machine precision — so when we say the NAMH figures reproduce, we mean bit-for-bit.

Case study: a gate that caught itself. The most instructive event of the release was a failure. The `soch_robustness` method re-runs the published SOCH direction-symmetry test over a nuisance grid (two thresholds $\times$ two filter orders, 112 grid points) on the eight advanced markets, with a bootstrap null. Its band was pre-registered from the project’s closure board: the baseline cell must hold all 28 market pairs, at an overall pass rate of at least 0.90. The first genuine end-to-end run returned 27/28 at baseline and an overall rate of 0.9821 — and the row went red. The cause was not the estimator but the harness: the driver carried no random-number-generator pin, so the bootstrap’s threshold was stochastic across runs and one near-boundary pair flipped between executions. The repair followed the engine’s seeded-determinism precedent — `pin RNGkind("L’Ecuyer-CMRG")` with seed 42 [7], and echo both in the output — and *the band was not touched*. A fresh anchor (`job_20260612_fe0c4f06`) returned 28/28 at a rate of 0.9911; an independent re-submission (`job_20260612_c0398d0f`) returned the identical tuple, digit for digit. The evaluation was then *tightened*, not widened: it now requires the exact seeded rate, the exact baseline count, and the seed itself to be echoed. The institutional lesson is that the gate which failed was the system working — a pre-registered band over a first-run driver surfaced a latent nondeterminism that would otherwise have produced intermittent, unexplainable reds in nightly operation.

5 The GPU upgrade

The transfer-entropy methods are the engine’s heaviest, and their cost lives in a single primitive: an $O(n^2)$ nearest-neighbour search under the Chebyshev (maximum) norm, repeated once per directed market pair and once per surrogate. On the workstation this work had a latent failure mode that, in the final NAMH tier deployment, brought the consequence to the surface: the host hung with all twenty-two logical cores pinned at one hundred per cent.

The hang, and the governance fix. The cause was specific and embarrassing in the way good bugs are. The transfer-entropy drivers hard-coded their worker count as `detectCores() - 1` — twenty-one forked workers — and ignored the core budget the job actually requested. Two concurrent robustness jobs therefore forked $2 \times 21 = 42$ workers over twenty-two cores, each worker sitting atop an unpinned multi-threaded BLAS, and the machine saturated and hung mid-pipeline. We removed the oversubscription at source: a single governed core budget, `ce_ncores()`, computed as `detectCores()` minus a reserve, overridden by the job’s request, and *clamped last* by a hard ceiling the job server sets to $\lfloor (\text{cpus} - 2) / \text{concurrency} \rfloor$; the server also pins the BLAS thread counts to one per spawned process. Worker count never changes a transfer-entropy number — the loops are order-free — so this is a determinism-safe change, and pinning the BLAS is itself a reproducibility gain. That makes the CPU path safe under concurrency. The complementary step was to move the search off the CPU entirely.

The offload, held to a bit-exact bar. The same primitive now runs on the workstation’s NVIDIA RTX 3000 Ada Laptop GPU through `numba.cuda` [10], a path that needed no new installation — the CUDA toolkit and `numba` were already present. The two methods dispatch to the GPU when a CUDA device is visible and fall back to the governed CPU path otherwise; a single environment switch (`CE_GPU=0`) forces the CPU path. The bar was the engine’s standing integrity rule: the GPU must *equal* the CPU, not approximate it. The observed transfer entropy — the only quantity the evaluation suite gates — is held bit-exact. A dedicated gate measured a maximum absolute deviation of 7×10^{-16} between the GPU and the engine’s own CPU estimator, with the six-decimal emitted figures identical, across embedding lags $\{1, 2\}$ and neighbour counts $\{4, 5\}$.

The gate also surfaced a floating-point subtlety worth recording, because it generalises. The engine’s one-dimensional neighbour count tests membership through the subtraction $v - \varepsilon$ rather than the textbook $|\Delta| < \varepsilon$, and so admits a boundary point that the strict form rejects when $|\Delta|$ rounds to exactly ε . We made the GPU reproduce the *engine’s* behaviour, not the textbook’s: when a published estimator is ported to a faster device, the equivalence test must be written against the estimator’s own output, floating-point conventions included, because the device that is “more correct” is the wrong device if it moves a number the literature has already recorded. Significance is unchanged in distribution — neither method seeds its generator, so the surrogate p -value is a non-reproducible draw in both paths — and the surrogate construction was vectorised into a batched Fourier transform on the host, so that a pair’s observed estimate and all of its surrogates now execute as one batched device launch and the per-launch overhead is paid once per pair rather than once per surrogate.

The test that mattered. The proof is the workload that hung the machine, re-run on the new path: the full eighteen-market panel, all 306 directed pairs, ninety-nine IAAFT surrogates each — 30,600 neighbour searches. It completed in 35.6 minutes at one hundred per cent of a *single* core, with a peak resident set of 388 MB, against a measured 289 seconds for a twelve-pair slice on the governed CPU path across roughly seven cores — which extrapolates to about two hours for the full panel. The GPU path is some 3.5× faster while leaving the processor, and the other twenty-one cores, free for concurrent work. The substantive result is unmoved: the

headline link, USA $\rightarrow$ Japan, returns transfer entropy 0.154234 to the digit; 106 of 306 directed links are significant at $p < 0.05$; and the strongest flows are all *into* Japan, recovering the same information-sink structure the CPU path reports, now as a by-product of a run that no longer threatens the host. The general lesson is the obvious one, stated plainly because it was learned the hard way: massively parallel fan-out belongs on hardware built for it, not on an unbounded CPU fork that competes with every other job on the machine.

6 Reproducibility, integrity and operations

The engine’s reliability is a property of its process, not a promise from its authors. Three commitments make that concrete.

Failable, pre-registered evaluations. Every method in the catalogue has an expected band fixed in advance of the run, and the evaluation suite is a single genuine execution of a committed runner whose only artefact is `evals.json`. We distinguish two kinds of red. A *band failure* means the engine is wrong: stop and investigate. An *extraction failure* means the harness misread the result’s shape: fix the check and re-run the whole suite — which is calibration, not gaming, only so long as the expected band is left untouched. `evals.json` is never hand-edited; it is always the output of one real run.

The nightly loop. The engine re-checks itself unattended. A reboot-surviving loop re-arms the job server if its port is closed (it never kills a running job), runs the full public suite with the async rows submitted as real tower jobs, refreshes the state ledger and appends any drift, refreshes the claims ledger, and — last, and deliberately nice’d so it can never contend with a tower job — renders an academic report of the night’s run and publishes it to the gallery. Each step is loud on failure and silent on success. A separate scheduled tick maintains the daily systemic-risk feed: it pulls the latest market closes, appends them to the stored panel, recomputes the index in a net-isolated step, and heals honestly over a missing day rather than fabricating one.

Formal and epistemic layers. Two layers sit above the numerical suite. A formal layer machine-checks the SOCH spectral results in Lean 4 with mathlib: eleven of twelve declarations are Lean-accepted, including the unconditional closure of the product-Lorentzian spectral-density proposition ($\|H(\omega)\|^2 = S(\omega)$); the single remaining declaration honestly carries a `sorry` and is reported as unproved — a `sorry` is never reported as proved. An epistemic layer maintains a provenance-carrying claims ledger spanning established, contested and superseded states, so that a contested result (for example, a channel-share figure that one method-and-partition choice puts higher than another) is displayed *as* contested, with both sides, rather than silently resolved.

The no-fabrication discipline. Underneath all of this is one rule the group applies to itself as strictly as to the engine: every number traces to a live computation or a documented published result, and every citation is looked up, never reconstructed from a codename. The discipline has teeth — it has caught fabricated bibliography titles and a mis-attributed arXiv identifier in our own drafts — and it is the reason a manual like this one can promise that each figure in it resolves to something checkable.

7 Outlook

The engine is deliberately small, and we intend to keep it so while widening what it can certify. Three directions are in view. First, more methods, on the same terms: a method joins the

catalogue only with a failable evaluation, and we would rather have twenty-six checked methods than fifty asserted ones. Second, a faster lane for the exploratory case: the GPU port runs in double precision to hold the published anchor bit-exact, but single precision is roughly two orders of magnitude faster again on this hardware and is the right default for interactive what-if analysis, provided it is clearly fenced off from the anchored figures. Third, and most ambitious, a move from a catalogue of methods towards a small set of shared computational primitives — long-memory estimation, wavelet decomposition, transfer entropy, surrogate generation, community detection — that several published frameworks instantiate, so that verifying the primitive verifies every method built on it. The throughline is unchanged from the first version of the engine to this one: a research result should be a re-runnable, self-checking object, and the infrastructure that makes it so is worth building once, carefully, and maintaining in the open.

References

- [1] Kraskov, A., Stögbauer, H., & Grassberger, P. (2004). Estimating mutual information. *Physical Review E*, 69(6), 066138.
- [2] Frenzel, S., & Pompe, B. (2007). Partial mutual information for coupling analysis of multivariate time series. *Physical Review Letters*, 99(20), 204101.
- [3] Schreiber, T. (2000). Measuring information transfer. *Physical Review Letters*, 85(2), 461–464.
- [4] Schreiber, T., & Schmitz, A. (1996). Improved surrogate data for nonlinearity tests. *Physical Review Letters*, 77(4), 635–638.
- [5] Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1), 289–300.
- [6] Diebold, F. X., & Yilmaz, K. (2012). Better to give than to receive: predictive directional measurement of volatility spillovers. *International Journal of Forecasting*, 28(1), 57–66.
- [7] L’Ecuyer, P. (1999). Good parameters and implementations for combined multiple recursive random number generators. *Operations Research*, 47(1), 159–164.
- [8] Bhandari, A., & Sahu (2026). *namh*: Network Adaptive Market Hypothesis estimators. R package version 0.1.0 (GPL-3).
- [9] Bhandari, A. (2026). Econstellar: an open-source AI-augmented research engine for computational financial econometrics. *arXiv:2606.05705*.
- [10] Lam, S. K., Pitrou, A., & Seibert, S. (2015). Numba: a LLVM-based Python JIT compiler. *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC ’15)*.